

PROGRAMMATION FONCTIONNELLE 2 : PARTIEL 4

juin 2026 — Pierre Rousselin

Exercice 1 : Preuves de logique des propositions (12 points)

Dans chacun des cas suivants, donner :

- une preuve Rocq en programmation fonctionnelle, c'est-à-dire avec un seul terme de preuve dans `exact` ;
- une preuve Rocq avec des tactiques de plus haut niveau et `exact` seulement à la fin sur une hypothèse ;
- un arbre de dérivation en déduction naturelle.

Ici, A, B et C sont trois propositions (en Rocq, des types de sorte `Prop`) quelconques.

1. (4 points) Lemma `ex4_1` : $((A \vee B) \rightarrow C) \rightarrow (A \rightarrow C) \wedge (B \rightarrow C)$.
 2. (4 points) Lemma `ex4_2` : $A \wedge \sim A \rightarrow B$. Pour le terme de preuve, on pourra utiliser la fonction `False_ind` dont le type est : `False_ind` : forall (P : Prop), False $\rightarrow$ P.
 3. (4 points) Lemma `ex4_3` : $\sim \sim \sim A \rightarrow \sim A$.
-

Exercice 2 : Arbres binaires colorés (15 points)

On considère le type inductif suivant, en Rocq.

```
(** *** Type ctree pour "Colored tree" : *)  
Inductive ctree (A : Type) :=  
  | cleaf (key : A)  
  | cnode (red : bool) (left right : ctree A).  
(* Le paramètre A est implicite dans les constructeurs : *)  
Arguments cleaf {A}. Arguments cnode {A}.
```

La particularité de ces arbres est que chaque nœud interne porte un drapeau booléen `red` (`true` si le nœud est « rouge », `false` sinon), tandis que les feuilles portent une étiquette dans A.

Par exemple, l'arbre ci-dessous, appelé `exple_5` par la suite (les nœuds rouges sont entourés), est un tel arbre, avec des étiquettes dans `nat` :

```
      (rouge)  
      /  \  
  (noir) leaf 30  
    /  \  
leaf 10 leaf 20
```

1. (1 point) Donner la définition en Rocq de `exple_5`.
2. (1 point) Inversement, représenter sous forme d'arbre (schéma, en indiquant la couleur de chaque nœud) l'arbre à étiquettes dans `string` :

```
Definition exple_6 :=
  cnode false (cleaf "a") (cnode true (cleaf "b") (cleaf "c")).
```

3. (1 point) On considère la fonction suivante, qui compte le nombre de nœuds rouges :

```
Fixpoint countRed {A : Type} (t : ctree A) : nat :=
  match t with
  | cleaf _      => 0                                (* crL *)
  | cnode r l r' => (if r then 1 else 0) + countRed l + countRed r' (* crN *)
  end.
```

Évaluer pas à pas `countRed exple_6` en étiquetant chaque étape de calcul par le commentaire correspondant dans la fonction.

4. (3 points) On considère la fonction suivante :

```
Fixpoint flipColors {A : Type} (t : ctree A) :=
  match t with
  | cleaf a      => cleaf a                                (* fl *)
  | cnode r l r' => cnode (negb r) (flipColors l) (flipColors r') (* fn *)
  end.
```

Donner la « preuve papier très détaillée » du lemme suivant (on rappelle que `negb (negb b) = b` pour tout booléen `b`, noté `negb_involutive`) :

```
Lemma flipColors_involutive {A : Type} (t : ctree A) :
  flipColors (flipColors t) = t.
```

5. (3 points) On considère les deux fonctions suivantes :

```
Fixpoint numLeaves {A : Type} (t : ctree A) :=
  match t with
  | cleaf _      => 1                                (* nLl *)
  | cnode _ l r  => numLeaves l + numLeaves r        (* nLn *)
  end.

Fixpoint toList {A : Type} (t : ctree A) : list A :=
  match t with
  | cleaf a      => [a]                                (* tLl *)
  | cnode _ l r  => toList l ++ toList r              (* tLn *)
  end.
```

Donner la « preuve papier très détaillée » du lemme suivant (on pourra utiliser sans preuve le fait que `length (l1 ++ l2) = length l1 + length l2`, noté `len_app`) :

```
Lemma length_toList {A : Type} (t : ctree A) :
  length (toList t) = numLeaves t.
```

6. (3 points) On considère la fonction suivante :

```
Fixpoint iter {A B : Type}
  (fl : A -> B) (fn : bool -> B -> B -> B) (t : ctree A) : B :=
  match t with
  | cleaf a      => fl a
```

```
| cnode r l r' => fn r (iter fl fn l) (iter fl fn r')
end.
```

Donner la « preuve papier très détaillée » du lemme suivant :

```
Lemma iter_numLeaves {A : Type} (t : ctree A) :
  iter (fun _ => 1) (fun _ x y => x + y) t = numLeaves t.
```

7. (3 points) En utilisant la fonction `iter`, sans récursivité (autre que celle déjà dans `iter`), définir :

- a) `countRed`, qui recalcule le nombre de nœuds rouges (comme `countRed` ci-dessus) ;
- b) `countBlack`, qui calcule le nombre de nœuds non rouges (« noirs ») ;
- c) `allLeaves p`, où $p : A \rightarrow \text{bool}$, qui vaut `true` si et seulement si toutes les feuilles de l'arbre satisfont le prédicat booléen p .

Exercice 3 : Prédicat inductif sur nat (13 points)

On se donne le prédicat inductif suivant (binaire sur nat) :

```
Inductive triple : nat -> nat -> Prop :=
| tz : triple 0 0
| ts (n m : nat) : triple n m -> triple (S n) (S (S (S m))).
```

1. (1 point) Représenter les deux constructeurs sous la forme de règles d'inférence (utiliser le nom du constructeur comme nom de la règle).
2. (4 points) Donner dans chaque cas suivant la preuve du fait demandé sous la forme d'un arbre de dérivation et en Rocq.
 - a) Lemma `triple26` : `triple 2 6`.
 - b) Lemma `triple412` : `triple 4 12`.

3. (4 points) On considère le lemme suivant :

```
Lemma triple_three (n : nat) : triple n (3 * n).
```

Le prouver par induction sur n , en indiquant dans chaque cas les hypothèses et la formule à prouver, et en terminant la preuve en Rocq.

4. (4 points) On considère le lemme et le début de preuve suivants :

```
Lemma triple_inv (n m : nat) : triple n m -> m = 3 * n.
Proof.
  intros H. induction H as [| n m Hnm IH].
```

Terminer la preuve, en donnant dans chaque cas les hypothèses et le but, puis la fin de la preuve en Rocq.

Exercice 4 : N, multiplication, listes (20 points)

On rappelle le type `N` modélisant les entiers naturels écrits en binaire :

```
Inductive N : Set := N0 : N | Npos (p : positive) : N.
```

On définit la multiplication sur N à partir de la multiplication Pos.mul sur positive (supposée déjà définie et correcte, ainsi que le fait que xH est neutre pour Pos.mul, noté Pos.mul_1_l : forall p, Pos.mul xH p = p) :

```
Definition Nmul (x y : N) : N :=
  match x, y with
  | N0, _      => N0
  | _, N0      => N0
  | Npos p, Npos q => Npos (Pos.mul p q)
  end.
```

```
Notation "x *N y" := (Nmul x y) (at level 40).
```

On suppose également connue et déjà prouvée l'associativité de *N :

```
Nmul_assoc : forall x y z, x *N (y *N z) = (x *N y) *N z.
```

1. (1 point) Donner la valeur de N0 *N Npos xH et de Npos xH *N N0.
2. (3 points) Prouver le lemme suivant :

```
Lemma Nmul_0_l (x : N) : N0 *N x = N0.
```

3. (5 points) Prouver le lemme suivant, en raisonnant par cas sur x :

```
Lemma Nmul_1_l (x : N) : Npos xH *N x = x.
```

On considère maintenant les listes de N, et la fonction suivante, qui calcule le produit des éléments d'une liste (la liste vide ayant pour produit l'élément neutre Npos xH) :

```
Fixpoint prodList (l : list N) : N :=
  match l with
  | []      => Npos xH
  | h :: t => h *N prodList t
  end.
```

4. (2 points) Sans donner de preuve, définir de la même manière la fonction sumList, qui calcule la somme des éléments d'une liste de N (on utilisera Nadd, défini dans les cours précédents, et l'élément neutre N0).
5. (9 points) Prouver le lemme suivant, en raisonnant par induction sur l1 :

```
Lemma prodList_app (l1 l2 : list N) :
  prodList (l1 ++ l2) = prodList l1 *N prodList l2.
```

Annexe 1 : règles de la déduction naturelle

$$\frac{}{\Gamma, A \vdash A} (Ax) \qquad \frac{\Gamma \vdash \text{False}}{\Gamma \vdash A} (\text{ex falso})$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (\wedge I) \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (\wedge E-g) \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} (\wedge E-d)$$

$$\begin{array}{c}
\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B}(\vee\text{I-g}) \quad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B}(\vee\text{I-d}) \quad \frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash P \quad \Gamma, B \vdash P}{\Gamma \vdash P}(\vee\text{E}) \\
\\
\frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B}(\rightarrow\text{I}) \quad \frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B}(\rightarrow\text{E}) \\
\\
\frac{\Gamma, x:T \vdash P}{\Gamma \vdash \forall x:T, P}(\forall\text{I}) \quad \frac{\Gamma \vdash \forall(x:T), P \quad \Gamma \vdash t:T}{\Gamma \vdash P\{x/t\}}(\forall\text{E})
\end{array}$$

Quelques définitions standard en Rocq

```

Inductive bool := true | false.
Definition andb b1 b2 := if b1 then b2 else false.
Definition orb  b1 b2 := if b1 then true  else b2.
Definition negb b := match b with true => false | false => true end.
Notation "b1 && b2" := (andb b1 b2).
Notation "b1 || b2" := (orb b1 b2).

Inductive and (A B : Prop) : Prop := conj (hA : A) (hB : B).
Inductive or  (A B : Prop) : Prop :=
  or_introl (hA : A) | or_intror (hB : B).

Inductive nat : Set := 0 : nat | S : nat -> nat.
(* Notations usuelles : 0 pour 0, 1 pour (S 0), etc *)
Fixpoint add (n m : nat) :=
  match n with
  | 0    => m
  | S p => S (add p m)
  end.

Inductive list (A : Type) : Type :=
  nil : list A | cons : A -> list A -> list A.
(* Notation [] pour nil et h :: t pour (cons h t) *)

Inductive positive : Set :=
  | xI (p : positive) | x0 (p : positive) | xH.
Notation "1"      := xH.
Notation "p ~ 1" := (xI p).
Notation "p ~ 0" := (x0 p).

```

Liste de tactiques Rocq vues en cours, TD et TP

- **exact** : terminer la preuve en cours avec un terme de preuve
- **intros** : introduire des variables ou hypothèses depuis le but vers le contexte local

- **apply** : appliquer une implication (dans le but ou, avec in, dans une hypothèse) en remplissant les trous avec de l'unification
- **exfalse** : remplacer le but par False
- **destruct** : raisonner par cas sur une valeur en liant des variables éventuelles aux arguments éventuels du ou des constructeurs
- **left ou right** : choisir le premier ou le deuxième constructeur ; plus souvent utilisé pour prouver une disjonction
- **split** : utiliser l'unique constructeur ; plus souvent utilisé pour prouver une conjonction
- **reflexivity** : terminer la preuve en cours lorsque le but se réduit à $a = a$
- **rewrite** : remplacer un terme par un autre qui lui est égal
- **induction** : comme destruct mais avec des hypothèses d'induction dans le(s) cas d'argument(s) dont le type est le même que la valeur sur laquelle on raisonne par induction
- **specialize** : donner des arguments à une fonction ou, ce qui revient au même, instancier un lemme
- **simpl** : calculer en utilisant les définitions
- **discriminate** : prouver False à partir d'une égalité impossible entre constructeurs distincts
- **injection** : extraire les égalités des arguments à partir d'une égalité entre constructeurs identiques
- **lia** : résoudre automatiquement les buts arithmétiques linéaires sur nat ou Z